Semester	B.E. Semester VII – Computer Engineering
Subject	DevOps Lab
Subject In-charge	Dr. Snehal Andhare

Student Name	Vicky Pukale
Roll Number	23102A0033
Grade and Subject Teacher's Signature	

Task Number	1
Project Title:	Secure CI/CD Pipeline for a Loan Application Tracking Portal
Task Title	Problem Definition and Scope: Study the real-time need for the Loan Application Tracking Portal. Identify target users, existing pain points, stakeholders, constraints and measurable success criteria. Freeze a small 15-Task MVP scope. Problem statement, stakeholder list, objectives, constraints and approved MVP scope.
Deliverable:	

1. Problem Statement

In traditional lending and financial operations, loan processing is plagued by opacity, prolonged turnaround times, and fragmented communication between applicants, loan officers, and credit verifiers:

- **For Borrowers:** Applicants face severe anxiety and uncertainty due to the absence of real-time visibility into their application stages (e.g., *Submitted*, *KYC Verified*, *Underwriting*, *Approved*, *Disbursed*, or *Rejected*). Inquiries often require manual follow-ups via phone or branch visits.
- **For Financial Institutions:** Loan officers and credit analysts waste operational hours handling repetitive status queries manually rather than evaluating credit risk. Fragmented systems increase the risk of miscommunication and data entry errors.
- **For Engineering & Operations:** Handling sensitive financial data (Personally Identifiable Information - PII, financial statements, PAN/SSN) requires stringent security compliance. Manual deployments and ad-hoc infrastructure setups introduce human errors, configuration drift, prolonged release cycles, and security vulnerabilities.

The Proposed Solution

Develop a lightweight, high-reliability **Loan Application Tracking Portal** coupled with an end-to-end, automated, and **Secure CI/CD Pipeline**:

1. **Application Layer:** Provides real-time event entry, search dashboards, status drill-down, and alerts for rejected or pending applications.
2. **DevOps Layer:** Enforces automated building, unit testing, Selenium automated regression testing, containerization via Docker, continuous deployment via Jenkins, and automated node provisioning using Ansible.

2. Target Users and Pain Points

User Persona	Role / Description	Current Pain Points	Desired Outcome
Loan Applicant (End User)	Individual applying for personal, home, or education loans.	No self-service tracking; unaware of missing documents or exact status; long waiting times.	Instant status search using Application ID; step-by-step progress tracker; clear alert if documents are rejected.
Loan Officer / Branch Staff	Bank employee intake and initial verification.	Burdened with repetitive inquiries; manual data recording; slow handoff to credit teams.	Unified dashboard to log application events, view application counts, and filter by status.
Credit Analyst / Underwriter	Risk assessment and approval authority.	Lack of single-pane view of pending vs. flagged loans; slow audit trail.	Rapid status drill-down; instant exception/risk flags; audit-ready event history.
DevOps / SysAdmin Engineer	Manages application lifecycle and infrastructure.	Manual deployment errors; security gaps in builds; slow turnaround for bug fixes; environment inconsistencies.	Automated Git-triggered CI/CD, containerized isolated deployments, zero-drift Ansible provisioning.

3. Stakeholder Register

1. **Project Sponsor / Product Owner:** Oversees business alignment, timely delivery, and operational compliance.
2. **Branch Operations & Underwriting Team:** Primary internal users validating daily workflows and operational utility.
3. **Information Security & Compliance Officer:** Ensures customer data protection, access controls, and pipeline security best practices.
4. **Development & QA Team:** Designs the web portal, builds features across branches, and maintains unit and Selenium UI test suites.
5. **DevOps Engineering Team:** Architects the CI/CD pipeline (Jenkins, Docker, GitHub, Ansible) and maintains server reliability.

4. Objectives and Measurable Success Criteria

Business & Functional Objectives

- **100% Self-Service Visibility:** Enable applicants to check real-time application status 24/7 without human intervention.
- **Turnaround Time (TAT) Reduction:** Reduce status update latency to sub-second updates upon event submission.
- **Auditability:** Every state transition (e.g., *Submitted* →→ *Document Verified* →→ *Approved*) must be timestamped and queryable.

DevOps & Engineering KPIs (DORA Metrics Aligned)

- **Lead Time for Changes:** Code commit in GitHub to deployment in a target container in under **10 minutes**.
- **Deployment Frequency:** On-demand, single-click, or automated continuous deployment upon successful pull request merge.
- **Automated Quality Gate: 100% pass rate** on critical Selenium user journeys prior to container release; automatic pipeline abort if tests fail.
- **Infrastructure as Code (IaC) Idempotency:** Ansible automation must be able to configure a clean server repeatedly with **zero errors** and zero configuration drift.
- **Zero-Downtime Rollback:** Recovery to the last stable Docker image within **2 minutes** in the event of a deployment fault.

5. Constraints and Assumptions

Constraints

1. **Security & Privacy:** Customer financial data and identification numbers must not be logged in cleartext inside CI/CD build logs.
2. **Environment & Resource Footprint:** The application and toolchain (Jenkins, Docker, Selenium, Ansible) must run within accessible lab/workstation limits (typically a Linux VM / Ubuntu 22.04 LTS or local dual-core/8GB RAM machine).
3. **Tech Stack Simplicity:** The MVP will prioritize clean, maintainable architecture (e.g., Java with Spring Boot / Maven or Python/Flask) packaged as a self-contained WAR/JAR or container image to keep build pipelines deterministic.

Assumptions

- The core development team has access to a GitHub repository, a Jenkins controller instance, and a local or VM-based target execution environment.
- The application uses an embedded or lightweight relational datastore (H2 / SQLite / PostgreSQL / MySQL) for the MVP baseline.

6. Approved 15-Task MVP Scope Definition

To ensure your project progresses systematically, here is the approved roadmap frozen for all 15 tasks:

Milestone / Task	Scope Summary	Key Deliverable
Task 1	**Problem Definition & Scope**	Problem statement, stakeholder list, objectives, constraints, frozen MVP scope document.
Task 2	**Agile Planning & DevOps Workflow**	User stories, acceptance criteria, product backlog, 15-task board, Definition of Done, DevOps lifecycle diagram.
Task 3	**Architecture & Tech Setup**	SRS summary, use-case diagram, architecture diagram, API/data model, local build environment setup.
Task 4	**Git & GitHub Initialization**	Repository creation, `.gitignore`, README, branch rules, issue templates, initial skeleton commit.

Task 5	**Feature 1 Branching Workflow**	`feature/event-entry` branch, Git CLI operations, PR creation, review, and merge to `dev`.
Task 6	**MVP Completion & Collaboration**	Complete remaining MVP features (search, dashboard, alerts), deliberate merge conflict resolution, release tagging (`v1.0.0`).
Task 7	**Jenkins Installation & CI Job**	Jenkins setup, GitHub webhook/polling connection, automated Maven build, and artifact archiving.
Task 8	**Pipeline as Code (Jenkinsfile)**	Declarative `Jenkinsfile` with build, test, package, parameterized environment deployment.
Task 9	**Selenium Test Automation**	Automated UI test suite for 3–5 user journeys with assertions, test data, and failure screenshot captures.
Task 10	**Continuous Testing Quality Gate**	Jenkins test report integration, pipeline failure upon defect injection, bug fix verification.
Task 11	**Dockerization & Container Lifecycle**	Multi-stage `Dockerfile`, image build, tag, port mapping, lifecycle commands (`run`, `stop`, `inspect`, `rm`).
Task 12	**Jenkins-Docker Continuous Delivery**	Versioned Docker image creation in pipeline, push to registry (Docker Hub/local), auto-deployment of fresh container.
Task 13	**Configuration Management (Ansible)**	Ansible inventory and YAML playbooks provisioning system packages, users, folders, Docker runtime, and firewall.
Task 14	**Automated Provisioning & Rollback**	Clean target node provisioning, idempotency validation, automated health checks, rollback testing.
Task 15	**End-to-End Release & Viva Prep**	Full commit-to-production demonstration, technical documentation, architecture guide, and viva defense guide.